

Contents

Binary tree cousins	1
Stack using DLL	6
LCA of N-ary tree	10
Lock node depending on its descendants	13
Intersection of multiple linked lists	16
Simple concurrent hashmap in java	20
Simple hashmap in java	25
Alternate Even Odd threads in java	28
Longest zero sequence between ones	31
Producer consumer threads in restaurant	32
Employees and reports under manager	39
Custom data type key value map	42
Calculate how many sentences can be created by replacing any word with one of its anagrams	47
Check if given binary tree is heap	49
Tic Tac Toe validator	52
Design water bottle	57
Create random game	63
Maximum Product Subarray	66
Unique Object	69
Thread creation	71

Binary tree cousins

```
import java.util.LinkedList;
import java.util.Queue;

public class BinaryTreeCousins {

    // Definition for a binary tree node.
    static class TreeNode {
```

```

    int val;

    TreeNode left, right;

    TreeNode(int val) {
        this.val = val;
        this.left = this.right = null;
    }
}

// Helper class to keep track of a node along with its parent.
static class NodeParent {
    TreeNode node;
    TreeNode parent;

    NodeParent(TreeNode node, TreeNode parent) {
        this.node = node;
        this.parent = parent;
    }
}

/**
 * Prints all cousins of the node whose value equals targetVal.
 * A "cousin" is any node at the same depth whose parent is different
 * from the target's parent.
 *
 * If the target node isn't found, or if it is the root, nothing is printed.
 */
public static void printCousins(TreeNode root, int targetVal) {
    if (root == null || root.val == targetVal) {
        // Empty tree or target is root → no cousins
        return;
    }

    // First BFS: locate target node's depth and parent
    Queue<NodeParent> queue = new LinkedList<>();
    queue.offer(new NodeParent(root, null));

```

```

int targetDepth = -1;

TreeNode targetParent = null;

int depth = 0;

boolean found = false;

while (!queue.isEmpty() && !found) {
    int levelSize = queue.size();
    for (int i = 0; i < levelSize; i++) {
        NodeParent np = queue.poll();
        TreeNode current = np.node;
        TreeNode parent = np.parent;

        if (current.val == targetVal) {
            targetDepth = depth;
            targetParent = parent;
            found = true;
            break;
        }
    }

    N-ary tree cousins
    for (Node child : current.children) {
        queue.offer(new NodeParent(child, current));
    }

    if (current.left != null) {
        queue.offer(new NodeParent(current.left, current));
    }
    if (current.right != null) {
        queue.offer(new NodeParent(current.right, current));
    }
}

if (!found) {
    depth++;
}

}

// If target not found or parent is null (i.e., target is root), exit
if (!found || targetParent == null) {

```

```

        return;
    }

    // Second BFS: traverse until we reach targetDepth, then print cousins
    queue.clear();
    queue.offer(new NodeParent(root, null));
    depth = 0;

    while (!queue.isEmpty()) {
        int levelSize = queue.size();

        if (depth == targetDepth) {
            // At target depth: print nodes whose parent ≠ targetParent
            for (int i = 0; i < levelSize; i++) {
                NodeParent np = queue.poll();
                TreeNode current = np.node;
                TreeNode parent = np.parent;
                if (parent != targetParent) {
                    System.out.print(current.val + " ");
                }
            }
            System.out.println();
            return; // done printing cousins
        }

        // Otherwise, descend one level deeper
        for (int i = 0; i < levelSize; i++) {
            NodeParent np = queue.poll();
            TreeNode current = np.node;

            N-ary tree cousins
            for (Node child : current.children) {
                queue.offer(new NodeParent(child, current));
            }

            if (current.left != null) {
                queue.offer(new NodeParent(current.left, current));
            }

            if (current.right != null) {

```

```

        queue.offer(new NodeParent(current.right, current));
    }
}
depth++;
}
}

```

// Example usage

```
public static void main(String[] args) {
```

```
    /*
```

```
    * Construct this sample binary tree:
```

```
    *
```

```
    *          (1)
```

```
    *        /    \
```

```
    *      (2)    (3)
```

```
    *     /  \   /  \
```

```
    *   (4) (5) (6) (7)
```

```
    *
```

```
    * For targetVal = 5, cousins are 6 and 7.
```

```
    */
```

```
    TreeNode root = new TreeNode(1);
```

```
    root.left = new TreeNode(2);
```

```
    root.right = new TreeNode(3);
```

```
    root.left.left = new TreeNode(4);
```

```
    root.left.right = new TreeNode(5);
```

```
    root.right.left = new TreeNode(6);
```

```
    root.right.right = new TreeNode(7);
```

```
    System.out.print("Cousins of 5: ");
```

```
    printCousins(root, 5); // Expected output: 6 7
```

```
    System.out.print("Cousins of 4: ");
```

```
    printCousins(root, 4); // Expected output: 6 7
```

```

        System.out.print("Cousins of 2: ");

        printCousins(root, 2); // Depth of 2 is 1; cousins at depth 1 with parent ≠ 1 → none.
    }
}

```

Stack using DLL

```

import java.util.HashMap;
import java.util.Map;

public class CustomStack<T> {
    /**
     * Each element in the stack is a Node<T> in a doubly-linked list.
     */
    public static class Node<T> {
        private T value;
        private Node<T> prev;
        private Node<T> next;

        private Node(T value) {
            this.value = value;
        }

        public T getValue() {
            return value;
        }
    }

    // Head points to the top of the stack
    private Node<T> head;
    private int size = 0;

    // Map from value → Node<T>, so we can delete by value in O(1).
    // This assumes each pushed value is unique. If you push duplicates,
    // you'd need a Map<T, List<Node<T>>> or similar.
    private final Map<T, Node<T>> nodeMap = new HashMap<>();
}

```

```

/**
 * Pushes a new value onto the stack and returns its Node<T> handle.
 * Stores that handle in the map so we can later do delete(value) in O(1).
 */
public Node<T> push(T value) {
    if (nodeMap.containsKey(value)) {
        throw new IllegalArgumentException(
            "Value already exists in stack: " + value);
    }

    Node<T> newNode = new Node<>(value);

    // Insert at head of DLL
    if (head != null) {
        head.prev = newNode;
        newNode.next = head;
    }

    head = newNode;
    size++;

    // Remember this node by its value
    nodeMap.put(value, newNode);
    return newNode;
}

/**
 * Pops and returns the top value. If the stack is empty, returns null.
 * Also removes it from the map.
 */
public T pop() {
    if (head == null) {
        return null;
    }

    T ret = head.value;

    // Remove head from the DLL
    Node<T> nextNode = head.next;

```

```

        if (nextNode != null) {
            nextNode.prev = null;
        }

        head.next = null; // help GC

        head = nextNode;

        size--;

        // Remove from the map
        nodeMap.remove(ret);

        return ret;
    }

    /**
     * Peeks at the top value without removing it.
     */
    public T peek() {
        return (head == null ? null : head.value);
    }

    /**
     * Deletes the node associated with the given value in O(1), if it exists.
     * If the value is not in the stack, does nothing (or you could throw).
     */
    public void delete(T value) {
        Node<T> node = nodeMap.get(value);

        if (node == null) {
            // Value not in stack, nothing to delete.

            return;
        }

        // If it's the head, just pop()

        if (node == head) {
            pop();

            return;
        }

        // Otherwise unlink in O(1)

```

```

        Node<T> prevNode = node.prev;

        Node<T> nextNode = node.next;

        if (prevNode != null) {
            prevNode.next = nextNode;
        }

        if (nextNode != null) {
            nextNode.prev = prevNode;
        }

        // Clean up pointers
        node.prev = null;
        node.next = null;
        size--;

        // Remove from map
        nodeMap.remove(value);
    }

    public boolean isEmpty() {
        return head == null;
    }

    public int size() {
        return size;
    }

    // ----- Example/Test -----

    public static void main(String[] args) {
        CustomStack<String> stack = new CustomStack<>();

        // Push some unique strings; map stores value → node
        stack.push("Alice");
        stack.push("Bob");
        stack.push("Charlie");
        stack.push("Diana");
    }

```

```

// Stack top → bottom: Diana, Charlie, Bob, Alice

System.out.println("Top is: " + stack.peek()); // Diana
System.out.println("Size is: " + stack.size()); // 4

// Delete "Charlie" by value in O(1):
stack.delete("Charlie");
// Now stack is: Diana (top), Bob, Alice

System.out.println("Size after deleting Charlie: " + stack.size()); // 3

// Pop off Diana:
System.out.println("Pop: " + stack.pop()); // Diana
// Pop off Bob:
System.out.println("Pop: " + stack.pop()); // Bob
// Pop off Alice:
System.out.println("Pop: " + stack.pop()); // Alice

System.out.println("Empty? " + stack.isEmpty()); // true
}
}

```

LCA of N-ary tree

```

/**
 * Utility class that provides a method to find the Lowest Common Ancestor
 * of two nodes in an N-ary tree.
 */
public class NaryTreeLCA {

    /**
     * Finds the LCA of nodes p and q in the tree rooted at 'root'.
     *
     * @param root the root of the N-ary tree
     * @param p one target node
     * @param q the other target node
     * @return the Node representing LCA(p, q), or null if neither p nor q
     * is found in this tree.
     */
}

```

```

*/

public static Node findLCA(Node root, Node p, Node q) {

    // Base cases:

    if (root == null) return null;

    if (root == p || root == q) {

        // If root matches one of the targets, return root.

        return root;

    }

    int countMatches = 0;

    Node tempResult = null;

    // Recurse into each child.

    for (Node child : root.children) {

        Node res = findLCA(child, p, q);

        if (res != null) {

            countMatches++;

            // Keep the non-null return in tempResult (if it's the first).

            if (tempResult == null) {

                tempResult = res;

            }

        }

        // If two children already reported back non-null, we can stop early.

        if (countMatches == 2) {

            return root;

        }

    }

    // If exactly one child subtree returned non-null, propagate it upward;

    // otherwise (zero or ≥2), return appropriately.

    return tempResult;

}

// --- Example usage / testing ---

public static void main(String[] args) {

    /*

```

```
* Build a sample N-ary tree:
```

```
*
```

```
*          (1)
```

```
*        /  |  \
```

```
*      (2) (3) (4)
```

```
*    /  \      / | \
```

```
*  (5) (6)  (7)(8)(9)
```

```
*
```

```
* LCA(5, 6) = 2
```

```
* LCA(5, 3) = 1
```

```
* LCA(7, 9) = 4
```

```
* LCA(8, 2) = 1
```

```
*/
```

```
Node root = new Node(1);
```

```
Node n2 = new Node(2);
```

```
Node n3 = new Node(3);
```

```
Node n4 = new Node(4);
```

```
root.children.add(n2);
```

```
root.children.add(n3);
```

```
root.children.add(n4);
```

```
n2.children.add(new Node(5));
```

```
n2.children.add(new Node(6));
```

```
n4.children.add(new Node(7));
```

```
n4.children.add(new Node(8));
```

```
n4.children.add(new Node(9));
```

```
// Let's get references to some nodes we want to test:
```

```
Node node5 = n2.children.get(0); // Node with val=5
```

```
Node node6 = n2.children.get(1); // Node with val=6
```

```
Node node3 = n3;                // Node with val=3
```

```
Node node7 = n4.children.get(0); // Node with val=7
```

```
Node node9 = n4.children.get(2); // Node with val=9
```

```
Node node8 = n4.children.get(1); // Node with val=8
```

```

        System.out.println("LCA(5,6) = " + findLCA(root, node5, node6).val);
        // expected 2

        System.out.println("LCA(5,3) = " + findLCA(root, node5, node3).val);
        // expected 1

        System.out.println("LCA(7,9) = " + findLCA(root, node7, node9).val);
        // expected 4

        System.out.println("LCA(8,2) = " + findLCA(root, node8, n2).val);
        // expected 1
    }
}

```

Lock node depending on its descendants

```

public class LockableNode {
    private LockableNode left;
    private LockableNode right;
    private LockableNode parent;

    // true if this node is locked
    private boolean isLocked = false;

    public LockableNode(LockableNode parent) {
        this.parent = parent;
    }

    /** Returns true if this node is currently locked. */
    public synchronized boolean isLocked() {
        return isLocked;
    }

    /**
     * Attempts to lock this node.
     * Only allowed if direct parent and direct children are not locked.
     * @return true if lock succeeded, false otherwise.
     */
}

```

```

*/

public synchronized boolean lock() {

    // If already locked, cannot lock again
    if (isLocked) return false;

    // Check direct parent
    if (parent != null && parent.isLocked()) return false;

    // Check direct children
    if ((left != null && left.isLocked()) || (right != null && right.isLocked())) {
        return false;
    }

    // All checks passed: lock this node
    isLocked = true;

    return true;
}

/**
 * Attempts to unlock this node.
 * @return true if unlock succeeded, false otherwise.
 */
public synchronized boolean unlock() {

    if (!isLocked) return false;

    isLocked = false;

    return true;
}

// Getters and setters for children, if needed
public LockableNode getLeft() {

    return left;
}

public LockableNode getRight() {

    return right;
}

public LockableNode getParent() {

    return parent;
}

```

```

public void setLeft(LockableNode left) {
    this.left = left;
    if (left != null) {
        left.parent = this;
    }
}

public void setRight(LockableNode right) {
    this.right = right;
    if (right != null) {
        right.parent = this;
    }
}

// Example usage / test
public static void main(String[] args) {
    /*
     * Build a small tree:
     *
     *      A
     *     / \
     *    B  C
     *   / \
     *  D  E
     */
    LockableNode A = new LockableNode(null);
    LockableNode B = new LockableNode(A);
    LockableNode C = new LockableNode(A);
    LockableNode D = new LockableNode(B);
    LockableNode E = new LockableNode(B);

    A.setLeft(B);
    A.setRight(C);
    B.setLeft(D);
    B.setRight(E);

    // Test locking logic with direct-level checks

```

```

        System.out.println("Lock D? " + D.lock()); // true

        System.out.println("Lock B? " + B.lock()); // false (child D is locked)

        System.out.println("Unlock D? " + D.unlock()); // true

        System.out.println("Lock B now? " + B.lock()); // true

        System.out.println("Lock D now? " + D.lock()); // false (parent B is locked)

        System.out.println("Unlock B? " + B.unlock()); // true

        System.out.println("Lock A? " + A.lock()); // true

        System.out.println("Lock C? " + C.lock()); // false (parent A is locked)

        System.out.println("Unlock A? " + A.unlock()); // true

        System.out.println("Lock C now? " + C.lock()); // true

    }
}

```

Intersection of multiple linked lists

```

import java.util.*;

/**
 * Definition for singly-linked list node.
 */
class ListNode {
    int val;
    ListNode next;
    ListNode(int x) {
        val = x;
        next = null;
    }
}

public class MergeGroupsFinder {
    /**
     * Given a list of head nodes (each representing a singly-linked list),
     * returns a list of groups (each group is a List<Integer> of indices)
     * corresponding to which input lists merge (share a tail) with each other.
     *
     * We assume:
     * - "heads" is a List of ListNode, where heads.get(i) is the head of the i-th linked list.
     * - Two lists are considered "merging" if they share any node by reference; that implies
     */
}

```

```

    * they have exactly the same tail node (since singly-linked lists cannot re-branch once
joined).

    * - We are NOT allowed to use extra space proportional to the total number of nodes ( $O(n)$  over
all nodes).

    * However, using  $O(L)$  extra space, where  $L$  = number of lists, is acceptable, since  $L \ll$  total
nodes.

    *

    * Time Complexity:  $O(N)$  where  $N$  = total number of nodes across all lists.
    * Space Complexity:  $O(L)$ , to store a map from tail node  $\rightarrow$  list of indices.
    */

public static List<List<Integer>> findMergingGroups(List<ListNode> heads) {
    // Map from tail node reference  $\rightarrow$  list of list-indices that share this tail
    Map<ListNode, List<Integer>> tailToIndices = new HashMap<>();

    for (int i = 0; i < heads.size(); i++) {
        ListNode head = heads.get(i);
        ListNode tail = getTail(head);

        // If the list is non-empty, group by its tail node
        if (tail != null) {
            tailToIndices.computeIfAbsent(tail, k -> new ArrayList<>()).add(i);
        }

        // If head is null (empty list), we can skip or treat separately—here we skip
    }

    // Collect only those groups that have size  $\geq 2$  (i.e., actually merge)
    List<List<Integer>> result = new ArrayList<>();
    for (List<Integer> group : tailToIndices.values()) {
        if (group.size()  $\geq 2$ ) {
            result.add(group);
        }
    }

    return result;
}

/**
    * Helper: traverse from head to find the last node (tail).
    * Returns null if head is null.
    * Time:  $O(\text{length of that list})$ . Across all lists, summation is  $O(\text{total nodes})$ .

```

```

*/

private static ListNode getTail(ListNode head) {
    if (head == null) return null;

    ListNode curr = head;

    while (curr.next != null) {
        curr = curr.next;
    }

    return curr;
}

// ----- Example Usage -----

public static void main(String[] args) {
    /*
     * Build an example with 5 lists, where:
     *
     * list 0: 1 → 2 → 3 ↘
     *
     *                7 → 8 → null
     *
     * list 1:      4 → 5 ↗
     *
     *
     * list 2: 10 → 11 → null
     *
     *
     * list 3: 20 → 30 → 40 ↘
     *
     *                8 → null    (8 is the same node as in lists 0 & 1)
     *
     *
     * list 4: null (empty)
     *
     *
     * So:
     *
     * - lists 0,1 share tail starting at node 7 (and thus merge).
     * - lists 0,1 also share node 8 at the very end, but tail is node 8 for all 3.
     * - list 3 also merges at node 8, so {0,1,3} all share tail node 8.
     * - list 2 is standalone (no merges).
     * - list 4 is empty, skip.
     *
     * Expected merging groups: [ [0,1,3] ]
     */

    // Shared tail: 7 → 8 → null

```

```

ListNode shared7 = new ListNode(7);
ListNode shared8 = new ListNode(8);
shared7.next = shared8;

// list0: 1→2→3→7→8
ListNode l0 = new ListNode(1);
l0.next = new ListNode(2);
l0.next.next = new ListNode(3);
l0.next.next.next = shared7;

// list1: 4→5→7→8
ListNode l1 = new ListNode(4);
l1.next = new ListNode(5);
l1.next.next = shared7;

// list2: 10→11→null
ListNode l2 = new ListNode(10);
l2.next = new ListNode(11);

// list3: 20→30→40→8→null
ListNode l3 = new ListNode(20);
l3.next = new ListNode(30);
l3.next.next = new ListNode(40);
l3.next.next.next = shared8;

// list4: null (empty)
ListNode l4 = null;

List<ListNode> heads = Arrays.asList(l0, l1, l2, l3, l4);
List<List<Integer>> groups = findMergingGroups(heads);

// Print result
System.out.println("Merging groups of list indices:");
for (List<Integer> grp : groups) {
    System.out.println(grp);
}

// Expected output: one group [0, 1, 3]

```

```
}  
}
```

Simple concurrent hashmap in java

```
import java.util.concurrent.locks.ReentrantLock;  
  
/**  
 * A simple thread-safe hash map implementation supporting concurrent get, put, and remove.  
 *  
 * Internally, it uses an array of buckets (linked lists of Nodes), and a parallel array of  
 * ReentrantLock objects—one lock per bucket. All operations on a given key acquire the lock  
 * for its bucket, so multiple threads can operate on different buckets in parallel.  
 *  
 * This implementation does NOT resize the table; it's meant as a minimal illustration of  
 * per-bucket locking. For production use, you'd want to add resizing (with careful re-locking  
 * or a global lock) or use java.util.concurrent.ConcurrentHashMap.  
 */  
public class ConcurrentHashMapCustom<K, V> {  
    /**  
     * A node in the singly-linked list for each bucket.  
     */  
    private static class Node<K, V> {  
        final K key;  
        volatile V value;  
        Node<K, V> next;  
  
        Node(K key, V value, Node<K, V> next) {  
            this.key = key;  
            this.value = value;  
            this.next = next;  
        }  
    }  
  
    // Default number of buckets; must be a power of two for bit masking.  
    private static final int DEFAULT_CAPACITY = 16;  
  
    /** Array of bucket heads. */
```

```

@SuppressWarnings("unchecked")

private final Node<K, V>[] table = (Node<K, V>[]) new Node[DEFAULT_CAPACITY];

/** One lock per bucket. */
private final ReentrantLock[] locks = new ReentrantLock[DEFAULT_CAPACITY];

public ConcurrentHashMapCustom() {
    for (int i = 0; i < DEFAULT_CAPACITY; i++) {
        locks[i] = new ReentrantLock();
    }
}

/**
 * Computes the bucket index for a given key, using bitmasking (capacity is power of two).
 */
private int bucketIndex(Object key) {
    int h = key.hashCode();

    // Spread bits (similar to java.util.HashMap)
    h ^= (h >>> 16);

    return h & (DEFAULT_CAPACITY - 1);
}

/**
 * Puts the (key, value) pair into the map. If the key already existed, replaces the old
 * value and returns it; otherwise returns null.
 * Thread-safe: locks only the target bucket.
 */
public V put(K key, V value) {
    if (key == null) throw new IllegalArgumentException("Null keys not supported");

    int idx = bucketIndex(key);
    ReentrantLock lock = locks[idx];

    lock.lock();

    try {
        Node<K, V> head = table[idx];

        // Search for existing key
        for (Node<K, V> curr = head; curr != null; curr = curr.next) {
            if (curr.key.equals(key)) {

```

```

        V old = curr.value;

        curr.value = value;

        return old;

    }

}

// Not found → insert new node at bucket head
Node<K, V> newNode = new Node<>(key, value, head);
table[idx] = newNode;

return null;
} finally {
    lock.unlock();
}
}

/**
 * Retrieves the value associated with the given key, or null if not present.
 * Thread-safe: locks only the target bucket during traversal.
 */
public V get(Object key) {
    if (key == null) return null;

    int idx = bucketIndex(key);
    ReentrantLock lock = locks[idx];
    lock.lock();

    try {
        for (Node<K, V> curr = table[idx]; curr != null; curr = curr.next) {
            if (curr.key.equals(key)) {
                return curr.value;
            }
        }

        return null;
    } finally {
        lock.unlock();
    }
}

/**
 * Removes the mapping for the given key if present, returning the old value, or null if none.

```

```

    * Thread-safe: locks only the target bucket.

    */

    public V remove(Object key) {
        if (key == null) return null;

        int idx = bucketIndex(key);

        ReentrantLock lock = locks[idx];

        lock.lock();

        try {
            Node<K, V> curr = table[idx];

            Node<K, V> prev = null;

            while (curr != null) {
                if (curr.key.equals(key)) {
                    // unlink curr

                    if (prev == null) {
                        table[idx] = curr.next;
                    } else {
                        prev.next = curr.next;
                    }

                    return curr.value;
                }

                prev = curr;
                curr = curr.next;
            }

            return null;
        } finally {
            lock.unlock();
        }
    }

}

/**
 * Returns true if the map contains no key-value mappings. This checks all buckets.
 * Thread-safe: locks each bucket in turn.
 */

    public boolean isEmpty() {
        for (int i = 0; i < DEFAULT_CAPACITY; i++) {
            locks[i].lock();

            try {

```

```

        if (table[i] != null) {
            return false;
        }
    } finally {
        locks[i].unlock();
    }
}

return true;
}

// Example usage / basic test
public static void main(String[] args) throws InterruptedException {
    ConcurrentHashMapCustom<String, Integer> map = new ConcurrentHashMapCustom<>();

    // Thread A: put some keys
    Thread tA = new Thread(() -> {
        for (int i = 0; i < 100; i++) {
            map.put("key" + i, i);
        }
    });

    // Thread B: read keys and print
    Thread tB = new Thread(() -> {
        for (int i = 0; i < 100; i++) {
            Integer v = map.get("key" + i);
            // Might be null if not yet inserted
            System.out.println("Get key" + i + " = " + v);
        }
    });

    // Thread C: remove some keys
    Thread tC = new Thread(() -> {
        for (int i = 50; i < 75; i++) {
            Integer old = map.remove("key" + i);
            System.out.println("Removed key" + i + " oldValue=" + old);
        }
    });
}

```

```

        tA.start();
        tB.start();
        tC.start();

        tA.join();
        tB.join();
        tC.join();

        System.out.println("Final check: key25=" + map.get("key25"));
        System.out.println("Final check: key60=" + map.get("key60")); // should be null
    }
}

```

Simple hashmap in java

```

import java.util.LinkedList;
import java.util.List;
import java.util.Objects;

public class SimpleHashMap<K,V> {
    // Initial number of buckets (must be a power of two for simplicity)
    private static final int INITIAL_CAPACITY = 16;

    // Load factor threshold before we resize
    private static final float LOAD_FACTOR = 0.75f;

    private List<Entry<K,V>>[] buckets;

    private int capacity;    // current number of buckets
    private int size;        // number of key/value pairs
    private int threshold;   // resize when size > threshold

    // Key/value pair
    private static class Entry<K,V> {
        final K key;

        V value;

        Entry(K k, V v) { key = k; value = v; }
    }
}

```

```

@SuppressWarnings("unchecked")

public SimpleHashMap() {

    this.capacity = INITIAL_CAPACITY;

    this.buckets = new LinkedList[capacity];

    for (int i = 0; i < capacity; i++) {

        buckets[i] = new LinkedList<>();

    }

    this.size = 0;

    this.threshold = (int)(capacity * LOAD_FACTOR);

}

/** Simple hash → bucket index */
private int bucketIndex(Object key) {

    int h = Objects.hashCode(key) & 0x7fffffff;

    return h % capacity;

}

/** Resize (double capacity) and rehash all entries */
@SuppressWarnings("unchecked")
private void resize() {

    int newCap = capacity * 2;

    List<Entry<K,V>>[] newBuckets = new LinkedList[newCap];

    for (int i = 0; i < newCap; i++) {

        newBuckets[i] = new LinkedList<>();

    }

    // Rehash entries

    for (List<Entry<K,V>> bucket : buckets) {

        for (Entry<K,V> e : bucket) {

            int idx = (Objects.hashCode(e.key) & 0x7fffffff) % newCap;

            newBuckets[idx].add(e);

        }

    }

    buckets = newBuckets;

    capacity = newCap;

    threshold = (int)(capacity * LOAD_FACTOR);

}

```

```

/** Put key→value, replacing if key already present */
public void put(K key, V value) {
    // Resize if loading above threshold
    if (size + 1 > threshold) {
        resize();
    }

    int idx = bucketIndex(key);
    List<Entry<K,V>> bucket = buckets[idx];
    for (Entry<K,V> e : bucket) {
        if (Objects.equals(e.key, key)) {
            e.value = value; // replace existing
            return;
        }
    }

    // not found: add new entry
    bucket.add(new Entry<>(key, value));
    size++;
}

/** Get the value for key, or null if missing */
public V get(K key) {
    int idx = bucketIndex(key);
    for (Entry<K,V> e : buckets[idx]) {
        if (Objects.equals(e.key, key)) {
            return e.value;
        }
    }

    return null;
}

/** Remove the key, returning its old value or null */
public V remove(K key) {
    int idx = bucketIndex(key);
    List<Entry<K,V>> bucket = buckets[idx];
    for (Entry<K,V> e : bucket) {
        if (Objects.equals(e.key, key)) {
            V old = e.value;

```

```

        bucket.remove(e);

        size--;

        return old;

    }

}

return null;
}

public int size() {
    return size;
}

// Simple test
public static void main(String[] args) {
    SimpleHashMap<String,Integer> map = new SimpleHashMap<>();
    for (int i = 0; i < 20; i++) {
        map.put("key"+i, i);
    }
    System.out.println("Size after inserts: " + map.size());
    System.out.println("key5 -> " + map.get("key5"));
    map.remove("key5");
    System.out.println("Removed key5, get(key5) -> " + map.get("key5"));
    System.out.println("Final size: " + map.size());
}
}

```

Alternate Even Odd threads in java

```

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class AlternatingEvenOddWithLock {
    private final int MAX;
    private int current = 1;

    private final Lock lock = new ReentrantLock();
    private final Condition oddTurn = lock.newCondition();

```

```

private final Condition evenTurn = lock.newCondition();

public AlternatingEvenOddWithLock(int max) {
    this.MAX = max;
}

public void printOdd() {
    while (true) {
        lock.lock();

        try {
            // If we've exceeded MAX, signal the other thread and break
            if (current > MAX) {
                evenTurn.signal();

                break;
            }

            // Wait while it's not odd's turn
            while (current <= MAX && current % 2 == 0) {
                oddTurn.await();
            }

            if (current > MAX) {
                evenTurn.signal();

                break;
            }

            // Print odd & increment
            System.out.println("OddThread: " + current);

            current++;

            evenTurn.signal(); // wake even thread
        } catch (InterruptedException ie) {
            Thread.currentThread().interrupt();

            break;
        } finally {
            lock.unlock();
        }
    }
}

```

```

}

public void printEven() {
    while (true) {
        lock.lock();

        try {
            if (current > MAX) {
                oddTurn.signal();
                break;
            }

            // Wait while it's not even's turn
            while (current <= MAX && current % 2 == 1) {
                evenTurn.await();
            }

            if (current > MAX) {
                oddTurn.signal();
                break;
            }

            // Print even & increment
            System.out.println("EvenThread: " + current);
            current++;

            oddTurn.signal(); // wake odd thread
        } catch (InterruptedException ie) {
            Thread.currentThread().interrupt();
            break;
        } finally {
            lock.unlock();
        }
    }
}

public static void main(String[] args) {
    final int MAX = 20;

    AlternatingEvenOddWithLock printer = new AlternatingEvenOddWithLock(MAX);

```

```

        Thread t1 = new Thread(printer::printOdd);
        Thread t2 = new Thread(printer::printEven);

        t1.start();
        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException ignored) { }

        System.out.println("-- Done printing up to " + MAX + " --");
    }
}

```

Longest zero sequence between ones

```

public class LongestZeroSequence {

    /**
     * Returns the length of the longest contiguous sequence of 0s that is surrounded by 1s
     * in the given array. A sequence is "surrounded" only if there is a 1 immediately before
     * it and a 1 immediately after it. Leading or trailing zeros (not enclosed by 1s) do not count.
     *
     * @param arr an array of integers containing only 0s and 1s
     * @return the maximum length of a zero-sequence that is sandwiched between two 1s
     */
    public static int longestZeroSequence(int[] arr) {
        int maxLen = 0;
        int currentLen = 0;
        boolean seenOne = false;

        for (int val : arr) {
            if (val == 1) {
                if (seenOne) {
                    // We have just encountered a 1 after counting zeros, so update max
                    maxLen = Math.max(maxLen, currentLen);
                }
                // Start (or restart) counting zeros after this 1
            }
        }
    }
}

```

```

        seenOne = true;

        currentLen = 0;
    } else { // val == 0

        if (seenOne) {

            // Only count zeros if we have seen a 1 to the left

            currentLen++;

        }

        // If we haven't seen a 1 yet, these zeros are leading and irrelevant
    }

}

return maxLen;

}

// Example usage and test cases
public static void main(String[] args) {

    int[][] tests = {

        {1, 0, 0, 1, 0, 0, 0, 1},

        {0, 0, 0, 0, 1},

        {1, 0, 0, 0, 0},

        {1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1},

        {1, 1, 0, 1, 0, 1, 0, 0, 1},

        {1, 0, 0, 1, 0, 1, 0, 0} // trailing zeros after last 1 should not count

    };

    for (int[] test : tests) {

        System.out.print("Array: ");

        for (int v : test) System.out.print(v + " ");

        System.out.println("\nLongest zero sequence surrounded by 1s: " +
longestZeroSequence(test));

        System.out.println("----");

    }

}

}

```

Producer consumer threads in restaurant

```
import java.util.LinkedList;
```

```

import java.util.Queue;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class RestaurantSimulation {

    // Simple class representing a raw Order
    static class Order {
        private final int id;

        Order(int id) { this.id = id; }

        public int getId() { return id; }
    }

    // Simple class representing a CookedOrder
    static class CookedOrder {
        private final int orderId;

        CookedOrder(int orderId) { this.orderId = orderId; }

        public int getOrderId() { return orderId; }
    }

    /**
     * Bounded buffer for raw orders, backed by ReentrantLock + Conditions.
     */
    static class OrderQueue {
        private final Queue<Order> queue = new LinkedList<>();
        private final int capacity;

        private final ReentrantLock lock = new ReentrantLock();
        private final Condition notFull = lock.newCondition();
        private final Condition notEmpty = lock.newCondition();

        public OrderQueue(int capacity) {
            this.capacity = capacity;
        }

        public void put(Order order) throws InterruptedException {
            lock.lock();

```

```

        try {

            // Wait if queue is full
            while (queue.size() == capacity) {

                notFull.await();

            }

            queue.offer(order);

            // Signal a waiting cook that there's something to take
            notEmpty.signal();

        } finally {

            lock.unlock();

        }

    }

}

public Order take() throws InterruptedException {

    lock.lock();

    try {

        // Wait if queue is empty
        while (queue.isEmpty()) {

            notEmpty.await();

        }

        Order order = queue.poll();

        // Signal a waiting order-taker that there's space
        notFull.signal();

        return order;

    } finally {

        lock.unlock();

    }

}

}

/**
 * Bounded buffer for cooked orders (to be served), using ReentrantLock + Conditions.
 */

static class CookedQueue {

    private final Queue<CookedOrder> queue = new LinkedList<>();

    private final int capacity;

```

```

private final ReentrantLock lock = new ReentrantLock();
private final Condition notFull = lock.newCondition();
private final Condition notEmpty = lock.newCondition();

public CookedQueue(int capacity) {
    this.capacity = capacity;
}

public void put(CookedOrder cookedOrder) throws InterruptedException {
    lock.lock();
    try {
        while (queue.size() == capacity) {
            notFull.await();
        }
        queue.offer(cookedOrder);
        // Signal a waiting server that there's something to serve
        notEmpty.signal();
    } finally {
        lock.unlock();
    }
}

public CookedOrder take() throws InterruptedException {
    lock.lock();
    try {
        while (queue.isEmpty()) {
            notEmpty.await();
        }
        CookedOrder order = queue.poll();
        // Signal a waiting cook that there's free space
        notFull.signal();
        return order;
    } finally {
        lock.unlock();
    }
}
}

```

```
// OrderTaker, Cook, Server, and main() remain unchanged except they now talk to the new queues:
```

```
static class OrderTaker implements Runnable {  
    private final OrderQueue orderQueue;  
    private final int id;  
    private static int nextOrderId = 1;  
  
    public OrderTaker(int id, OrderQueue orderQueue) {  
        this.id = id;  
        this.orderQueue = orderQueue;  
    }  
  
    @Override  
    public void run() {  
        try {  
            while (true) {  
                int orderId;  
                synchronized (OrderTaker.class) {  
                    orderId = nextOrderId++;  
                }  
                Order order = new Order(orderId);  
                orderQueue.put(order);  
                System.out.println("OrderTaker-" + id + " took Order #" + orderId);  
                Thread.sleep((long) (Math.random() * 500));  
            }  
        } catch (InterruptedException e) {  
            Thread.currentThread().interrupt();  
        }  
    }  
}
```

```
static class Cook implements Runnable {  
    private final OrderQueue orderQueue;  
    private final CookedQueue cookedQueue;  
    private final int id;
```

```

public Cook(int id, OrderQueue orderQueue, CookedQueue cookedQueue) {

    this.id = id;

    this.orderQueue = orderQueue;

    this.cookedQueue = cookedQueue;

}

@Override

public void run() {

    try {

        while (true) {

            Order order = orderQueue.take();

            System.out.println("Cook-" + id + " got Order #" + order.getId() + " to cook");

            Thread.sleep((long) (Math.random() * 1000));

            CookedOrder cookedOrder = new CookedOrder(order.getId());

            cookedQueue.put(cookedOrder);

            System.out.println("Cook-" + id + " cooked Order #" + order.getId());

        }

    } catch (InterruptedException e) {

        Thread.currentThread().interrupt();

    }

}

}

static class Server implements Runnable {

    private final CookedQueue cookedQueue;

    private final int id;

    public Server(int id, CookedQueue cookedQueue) {

        this.id = id;

        this.cookedQueue = cookedQueue;

    }

    @Override

    public void run() {

        try {

            while (true) {

                CookedOrder cookedOrder = cookedQueue.take();

```

```

        System.out.println("Server-" + id + " serving Cooked Order #" +
        cookedOrder.getOrderID());

        Thread.sleep((long) (Math.random() * 500));

    }

    } catch (InterruptedException e) {

        Thread.currentThread().interrupt();

    }

    }

}

public static void main(String[] args) {

    final int ORDER_QUEUE_CAPACITY = 5;

    final int COOKED_QUEUE_CAPACITY = 5;

    OrderQueue orderQueue = new OrderQueue(ORDER_QUEUE_CAPACITY);

    CookedQueue cookedQueue = new CookedQueue(COOKED_QUEUE_CAPACITY);

    for (int i = 1; i <= 3; i++) {

        Thread t = new Thread(new OrderTaker(i, orderQueue));

        t.setDaemon(true);

        t.start();

    }

    for (int i = 1; i <= 2; i++) {

        Thread t = new Thread(new Cook(i, orderQueue, cookedQueue));

        t.setDaemon(true);

        t.start();

    }

    for (int i = 1; i <= 2; i++) {

        Thread t = new Thread(new Server(i, cookedQueue));

        t.setDaemon(true);

        t.start();

    }

    try {

        Thread.sleep(10_000);

    } catch (InterruptedException e) {

        Thread.currentThread().interrupt();

    }

```

```

    }

    System.out.println("Simulation finished.");

}
}

```

Employees and reports under manager

```

import java.util.*;

/**
 * Given a list of employees with (name, managerName, salary),
 * build an in-memory structure to quickly look up all employees
 * reporting (directly or indirectly) to a given manager.
 */
public class EmployeeHierarchy {

    /** Simple DTO for an employee record. */
    static class Employee {

        String name;

        String managerName; // null or empty if top-level (no manager)

        double salary;

        public Employee(String name, String managerName, double salary) {

            this.name = name;

            this.managerName = managerName;

            this.salary = salary;

        }

    }

    /**
     * Builds a map: managerName → list of direct reports (Employee objects).
     *
     * @param employees the full list of employees
     * @return a map where key = manager name, value = list of Employees who report directly
     */
    private static Map<String, List<Employee>> buildManagerMap(List<Employee> employees) {

        Map<String, List<Employee>> mgrToReports = new HashMap<>();
    }
}

```

```

    for (Employee e : employees) {
        String mgr = e.managerName;
        if (mgr == null || mgr.isEmpty()) {
            // top-level (no manager) employees can be skipped or stored under a special key
            continue;
        }

        mgrToReports
            .computeIfAbsent(mgr, k -> new ArrayList<>())
            .add(e);
    }

    return mgrToReports;
}

/**
 * Given a manager name and the map of direct reports, returns the
 * names of all employees who report (directly or indirectly) to that manager.
 *
 * Uses a DFS (stack) or BFS (queue) to traverse the tree under the given manager.
 *
 * @param managerName the name of the manager whose subtree we want
 * @param mgrToReports map built by buildManagerMap(...)
 * @return a List<String> of all employee names under that manager (excluding the manager
 *         himself)
 */
public static List<String> getAllReports(String managerName,
                                         Map<String, List<Employee>> mgrToReports) {

    List<String> result = new ArrayList<>();

    if (managerName == null || managerName.isEmpty()) {
        return result;
    }

    // Use a queue for BFS
    Deque<String> queue = new ArrayDeque<>();
    queue.add(managerName);

    while (!queue.isEmpty()) {

```

```

        String mgr = queue.poll();

        List<Employee> directReports = mgrToReports.get(mgr);

        if (directReports == null) {
            continue;
        }

        for (Employee emp : directReports) {
            result.add(emp.name);
            queue.add(emp.name);
        }
    }

    return result;
}

/** Example usage */
public static void main(String[] args) {
    List<Employee> employees = List.of(
        new Employee("Alice",    null,    200_000),    // CEO, no manager
        new Employee("Bob",      "Alice", 150_000),
        new Employee("Carol",    "Alice", 150_000),
        new Employee("David",    "Bob",   120_000),
        new Employee("Eve",      "Bob",   120_000),
        new Employee("Frank",    "Carol", 120_000),
        new Employee("Grace",    "David", 100_000),
        new Employee("Heidi",    "Eve",   100_000),
        new Employee("Ivan",     "Frank", 100_000),
        new Employee("Judy",     "Frank", 100_000)
    );

    // Build the manager → direct reports map
    Map<String, List<Employee>> mgrToReports = buildManagerMap(employees);

    // Suppose we want all reports (direct & indirect) under "Bob"
    String queryManager = "Bob";

    List<String> reportsToBob = getAllReports(queryManager, mgrToReports);

    System.out.println("All employees reporting to " + queryManager + ":");

    for (String name : reportsToBob) {

```

```

        System.out.println("  " + name);
    }

    // Expected output:
    //   David
    //   Eve
    //   Grace      (reports to David)
    //   Heidi      (reports to Eve)

    // Similarly, all reports under "Alice"
    queryManager = "Alice";

    List<String> reportsToAlice = getAllReports(queryManager, mgrToReports);

    System.out.println("\nAll employees reporting to " + queryManager + ":");

    for (String name : reportsToAlice) {
        System.out.println("  " + name);
    }

    // Expected output:
    //   Bob
    //   Carol
    //   David
    //   Eve
    //   Frank
    //   Grace
    //   Heidi
    //   Ivan
    //   Judy
}
}

```

Custom data type key value map

```

import java.util.*;

/**
 * A Trie (prefix-tree) implementation for string keys consisting of digits '0'-'9'.
 * This supports get, put, and delete in O(L) time, where L is the length of the key,
 * independent of the number of stored keys. It handles arbitrarily long numeric keys
 * (OBJ_IDs that exceed primitive ranges).
 */

```

```

* Usage:

*   TrieMap<String> map = new TrieMap<>();
*   map.put("12345678901234567890", "Alice");
*   String v = map.get("12345678901234567890"); // "Alice"
*   map.delete("12345678901234567890");
*   v = map.get("12345678901234567890"); // null
*/

public class TrieMap<V> {

    private static final int RADIX = 10; // digits '0'-'9'

    /** Trie node */
    private static class Node<V> {

        // Children for digits '0'..'9'
        Node<V>[] children;

        // Stored value if this node marks the end of a key
        V value;

        // Indicates whether a key terminates exactly here
        boolean isEnd;

        @SuppressWarnings("unchecked")
        Node() {
            children = (Node<V>[]) new Node[RADIX];
            isEnd = false;
            value = null;
        }
    }

    private final Node<V> root = new Node<>();

    /** Inserts or updates the given numeric key with the given value. */
    public void put(String key, V value) {
        Node<V> node = root;
        for (char c : key.toCharArray()) {
            int d = c - '0';
            if (d < 0 || d >= RADIX) {
                throw new IllegalArgumentException("Key contains non-digit character: " + c);
            }
        }
    }
}

```

```

        if (node.children[d] == null) {
            node.children[d] = new Node<>();
        }
        node = node.children[d];
    }
    node.isEnd = true;
    node.value = value;
}

/** Returns the value associated with key, or null if not present. */
public V get(String key) {
    Node<V> node = root;
    for (char c : key.toCharArray()) {
        int d = c - '0';
        if (d < 0 || d >= RADIX) return null;
        if (node.children[d] == null) return null;
        node = node.children[d];
    }
    return node.isEnd ? node.value : null;
}

/**
 * Deletes the key (and its associated value) if present.
 * Returns true if a key was deleted, false if key was not found.
 */
public boolean delete(String key) {
    return delete(root, key, 0);
}

/**
 * Recursively deletes the key from the trie.
 * Returns true if the caller should prune this node (no children and not isEnd).
 */
private boolean delete(Node<V> node, String key, int idx) {
    if (idx == key.length()) {
        if (!node.isEnd) {
            // Key does not exist

```

```

        return false;
    }

    // Unset this node's end-of-key flag
    node.isEnd = false;
    node.value = null;

    // If no children, tell parent to prune
    return noChildren(node);
}

int d = key.charAt(idx) - '0';
if (d < 0 || d >= RADIX || node.children[d] == null) {
    // Key not found
    return false;
}

boolean shouldPruneChild = delete(node.children[d], key, idx + 1);
if (shouldPruneChild) {
    node.children[d] = null;
    // If this node also is not end of another key and has no other children, prune it
    return !node.isEnd && noChildren(node);
}

return false;
}

/** Helper: returns true if node has no children. */
private boolean noChildren(Node<V> node) {
    for (Node<V> child : node.children) {
        if (child != null) {
            return false;
        }
    }

    return true;
}

/** Returns true if the key is present in the trie. */
public boolean containsKey(String key) {
    Node<V> node = root;

    for (char c : key.toCharArray()) {
        int d = c - '0';

```

[illegible]

```
map.delete("000000000000000000000000")); // false  
  
}  
  
}
```

Calculate how many sentences can be created by replacing any word with one of its anagrams

Note:

Two words are said to be anagrams of each other if one can be created by rearranging the letters of another word, using all the original letters exactly once.

```
wordSet = ['listen', 'silent', 'it', 'is']  
sentences = 'listen it is silent', 'silent it is silent'
```

```
wordSet = ['cat', 'act', 'is']  
sentences = 'cat is tac', 'act tac is cat'
```

Write the `countSentences` function in the editor below.

```
countSentences has the following parameters:
```

- string wordSet[n]: an array of strings
- string sentences[m]: an array of strings

Returns:

- `int[]`: an array of `m` integers that denote the number of sentences that can be formed from each sentence

```
import java.util.*;
```

```
public class Solution {
```

/ **

* For each sentence, counts how many distinct sentences can be formed by replacing

```
* each word with any of its anagrams from the given wordSet.
```

```
* @param wordSet    array of strings representing the dictionary of words
```

```
* @param sentences array of sentences (each sentence is a space-separated string)
```

```

* @return an array of integers where each element is the count for the corresponding sentence
*/

public static int[] countSentences(String[] wordSet, String[] sentences) {
    // Build a map: canonical form (sorted letters) → frequency of words in wordSet
    Map<String, Integer> anagramCount = new HashMap<>();

    for (String w : wordSet) {
        String key = canonical(w);
        anagramCount.put(key, anagramCount.getOrDefault(key, 0) + 1);
    }

    int m = sentences.length;
    int[] result = new int[m];

    for (int i = 0; i < m; i++) {
        String sentence = sentences[i];
        String[] words = sentence.split("\\s+");
        long ways = 1L;
        for (String w : words) {
            String key = canonical(w);
            int freq = anagramCount.getOrDefault(key, 0);
            if (freq == 0) {
                ways = 0L;
                break;
            }
            ways *= freq;

            // If the product exceeds Integer.MAX_VALUE, clamp to that (assuming constraints)
            if (ways > Integer.MAX_VALUE) {
                ways = Integer.MAX_VALUE;
            }
        }
        result[i] = (int) ways;
    }

    return result;
}

// Helper: returns the canonical form of a word by sorting its characters
private static String canonical(String s) {

```

```

        char[] arr = s.toCharArray();

        Arrays.sort(arr);

        return new String(arr);
    }

    // Example usage
    public static void main(String[] args) {

        String[] wordSet1 = {"listen", "silent", "it", "is"};

        String[] sentences1 = {"listen it is silent", "silent it is silent"};

        int[] ans1 = countSentences(wordSet1, sentences1);

        // Expected: [4, 4]

        System.out.println(Arrays.toString(ans1));

        String[] wordSet2 = {"cat", "act", "is"};

        String[] sentences2 = {"cat is tac", "act tac is cat"};

        int[] ans2 = countSentences(wordSet2, sentences2);

        // "cat is tac" => ['cat'|'act'] × ['is'] × ['cat'|'act'] = 2 × 1 × 2 = 4

        // "act tac is cat" => 2 × 2 × 1 × 2 = 8

        System.out.println(Arrays.toString(ans2));

    }
}

```

Check if given binary tree is heap

```

// Java Program to Check if a given Binary
// Tree is a Heap

class Node {

    int key;

    Node left;

    Node right;

    Node(int k) {

        key = k;

        left = null;

        right = null;

    }

}

```

```

class GfG {

    // This function counts the number of nodes
    // in a binary tree
    static int countNodes(Node root) {
        if (root == null)
            return 0;

        return 1 + countNodes(root.left) + countNodes(root.right);
    }

    // This function checks if the binary tree is complete or not
    static boolean isCompleteUtil(Node root, int index, int numberOfNodes) {
        if (root == null)
            return true;

        // If index assigned to current node is more than
        // number of nodes in the tree, then the tree is not complete
        if (index >= numberOfNodes)
            return false;

        // Recur for left and right subtrees
        return isCompleteUtil(root.left, 2 * index + 1, numberOfNodes) &&
            isCompleteUtil(root.right, 2 * index + 2, numberOfNodes);
    }

    // This function checks the heap property in the tree.
    static boolean isHeapUtil(Node root) {
        if (root.left == null && root.right == null)
            return true;

        // Node will be in the second-last level
        if (root.right == null) {

            // Check heap property at the node
            // No recursive call because no need to
            // check the last level

```

```

        return root.key >= root.left.key;
    } else {

        // Check heap property at the node and recursively
        // check the heap property at left and right subtrees
        if (root.key >= root.left.key && root.key >= root.right.key)
            return isHeapUtil(root.left) && isHeapUtil(root.right);
        else
            return false;
    }
}

```

```

// Function to check if the binary tree is a Heap or not.
static boolean isHeap(Node root) {
    int nodeCount = countNodes(root);
    int index = 0;

    if (isCompleteUtil(root, index, nodeCount) && isHeapUtil(root))
        return true;
    return false;
}

```

```

public static void main(String[] args) {

```

```

    // Construct the Binary Tree

```

```

    //      10
    //     / \
    //    9   8
    //   / \ / \
    //  7  6 5  4

```

```

    Node root = new Node(10);
    root.left = new Node(9);
    root.right = new Node(8);
    root.left.left = new Node(7);
    root.left.right = new Node(6);
    root.right.left = new Node(5);

```

```

        root.right.right = new Node(4);

        if (isHeap(root)) {
            System.out.println("true");
        } else {
            System.out.println("false");
        }
    }
}

```

Tic Tac Toe validator

```

import java.util.*;

public class TicTacToeValidator {

    /**
     * Given an m×n Tic-Tac-Toe board (matrix of chars 'X','O',' '),
     * determines whether it is a valid state reachable by playing moves
     * alternately starting with 'X' until someone wins or board fills.
     *
     * We check:
     * 1) countX == countO or countX == countO + 1 (because X goes first).
     * 2) If X has any winning line, then countX == countO + 1.
     * 3) If O has any winning line, then countX == countO.
     * 4) X and O cannot both have winning lines.
     *
     * A “winning line” is defined as:
     * - Any full row of the same symbol,
     * - Any full column of the same symbol,
     * - If the board is square (m == n), the main diagonal or anti-diagonal.
     *
     * @param board an m×n char matrix, where each entry is 'X', 'O', or ' ' (space).
     * @return true if the configuration is reachable by a sequence of legal moves.
     */
    public static boolean validTicTacToe(char[][] board) {
        int m = board.length;

        if (m == 0) return false;
    }
}

```

```

int n = board[0].length;

// Count X's and O's
int countX = 0, countO = 0;
for (int i = 0; i < m; i++) {
    if (board[i].length != n) {
        throw new IllegalArgumentException("All rows must have the same length");
    }
    for (int j = 0; j < n; j++) {
        char c = board[i][j];
        if (c == 'X') countX++;
        else if (c == 'O') countO++;
        else if (c != ' ') {
            // invalid character
            return false;
        }
    }
}

// Rule 1: X goes first → countX == countO or countX == countO + 1
if (!(countX == countO || countX == countO + 1)) {
    return false;
}

boolean xWins = isWinner(board, 'X');
boolean oWins = isWinner(board, 'O');

// Rule 4: Both cannot win simultaneously
if (xWins && oWins) {
    return false;
}

// Rule 2: If X wins, X must have played one more move than O
if (xWins && countX != countO + 1) {
    return false;
}

// Rule 3: If O wins, X and O must have played the same number of moves
if (oWins && countX != countO) {
    return false;
}

```

```

    }

    return true;
}

/**
 * Checks if the given player ('X' or 'O') has any winning line on an mxn board.
 * We look for:
 * - any row completely filled with that player,
 * - any column completely filled,
 * - if m == n, the main diagonal or anti-diagonal completely filled.
 */
private static boolean isWinner(char[][] board, char player) {
    int m = board.length;
    int n = board[0].length;

    // Check each row
    for (int i = 0; i < m; i++) {
        boolean fullRow = true;
        for (int j = 0; j < n; j++) {
            if (board[i][j] != player) {
                fullRow = false;
                break;
            }
        }
        if (fullRow) return true;
    }

    // Check each column
    for (int j = 0; j < n; j++) {
        boolean fullCol = true;
        for (int i = 0; i < m; i++) {
            if (board[i][j] != player) {
                fullCol = false;
                break;
            }
        }
    }
}

```

```

        if (fullCol) return true;
    }

    // If square, check main diagonal
    if (m == n) {
        boolean mainDiag = true;
        for (int i = 0; i < m; i++) {
            if (board[i][i] != player) {
                mainDiag = false;
                break;
            }
        }
        if (mainDiag) return true;

        // Check anti-diagonal
        boolean antiDiag = true;
        for (int i = 0; i < m; i++) {
            if (board[i][n - 1 - i] != player) {
                antiDiag = false;
                break;
            }
        }
        if (antiDiag) return true;
    }

    return false;
}

// Example usage
public static void main(String[] args) {
    // Example 1: 3x3 board (same as before)
    char[][] board1 = {
        {'X', 'O', 'X'},
        {' ', 'X', ' '},
        {' ', ' ', 'O'}
    };

    System.out.println(validTicTacToe(board1)); // true
}

```

```

// Example 2: Invalid both-win case
char[][] board2 = {
    {'X', 'X', 'X'},
    {'O', 'O', 'O'},
    {' ', ' ', ' '}};

System.out.println(validTicTacToe(board2)); // false

// Example 3: 4x3 board (rectangular), no diagonals checked
char[][] board3 = {
    {'X', 'O', 'X'},
    {'O', 'X', 'O'},
    {'X', 'O', ' '},
    {' ', 'X', ' '}};

// CountX=5, CountO=4 → OK so far. No full row or column for either → valid.
System.out.println(validTicTacToe(board3)); // true

// Example 4: 4x4 board with a diagonal win for X
char[][] board4 = {
    {'X', 'O', ' ', ' '},
    {'O', 'X', 'O', ' '},
    {' ', 'O', 'X', 'O'},
    {' ', ' ', 'O', 'X'}};

// This has X on the main diagonal for a 4x4 → (0,0),(1,1),(2,2),(3,3)
// CountX = 4, CountO = 4 → O just moved. But X already has a win → invalid, because X must
have played last.

System.out.println(validTicTacToe(board4)); // false

// Example 5: 5x5 board with a full column of O
char[][] board5 = {
    {'X', 'O', ' ', ' ', ' '},
    {'O', 'O', 'X', ' ', ' '},
    {'X', 'O', ' ', 'O', ' '},
    {' ', 'O', 'X', 'X', ' '},
    {' ', 'O', ' ', ' ', ' '}};

```

```

        {'X', 'O', ' ', ' ', ' ', 'O'}

    };

    // Column 1 is all 'O' (i=0..4, j=1). O has a winning column.

    // CountX = 5, CountO = 6 → O must have played last → valid.

    System.out.println(validTicTacToe(board5)); // true

}
}

```

Design water bottle

```

// File: Bottle.java

public class Bottle {

    // Encapsulated fields: private visibility + getters/setters

    private String shape;          // e.g. "Cylindrical", "Square", "Oval"

    private String color;          // e.g. "Blue", "Transparent", "Red"

    private String material;       // e.g. "Plastic", "Stainless Steel", "Glass"

    private double capacityML;     // capacity in milliliters

    private CapType capType;      // way of opening/closing


    /**
     * How the cap works. Different bottle types might have different caps.
     * We model it as an enum here.
     */

    public enum CapType {

        SCREW_ON,

        FLIP_TOP,

        SPORT_CAP,    // push-pull spout

        STRAW_LID,

        CORK

    }


    // Constructor is protected so only subclasses (or package) can directly instantiate
    protected Bottle(String shape, String color, String material, double capacityML, CapType capType)
    {

        this.shape = shape;

        this.color = color;

        this.material = material;

        this.capacityML = capacityML;
    }
}

```

```
        this.capType = capType;
    }

    // Getters and setters (encapsulation)
    public String getShape() {
        return shape;
    }
    public void setShape(String shape) {
        this.shape = shape;
    }

    public String getColor() {
        return color;
    }
    public void setColor(String color) {
        this.color = color;
    }

    public String getMaterial() {
        return material;
    }
    public void setMaterial(String material) {
        this.material = material;
    }

    public double getCapacityML() {
        return capacityML;
    }
    public void setCapacityML(double capacityML) {
        if (capacityML <= 0) {
            throw new IllegalArgumentException("Capacity must be positive");
        }
        this.capacityML = capacityML;
    }

    public CapType getCapType() {
        return capType;
    }
}
```

```
}

public void setCapType(CapType capType) {
    this.capType = capType;
}

// Common behaviors
public void openCap() {
    switch (capType) {
        case SCREW_ON:
            System.out.println("Unscrewing the cap...");
            break;
        case FLIP_TOP:
            System.out.println("Flipping the top open...");
            break;
        case SPORT_CAP:
            System.out.println("Pulling up the sport cap...");
            break;
        case STRAW_LID:
            System.out.println("Pushing out the straw...");
            break;
        case CORK:
            System.out.println("Pulling out the cork...");
            break;
    }
}

public void closeCap() {
    switch (capType) {
        case SCREW_ON:
            System.out.println("Screwing the cap back on...");
            break;
        case FLIP_TOP:
            System.out.println("Flipping the top closed...");
            break;
        case SPORT_CAP:
            System.out.println("Pushing down the sport cap...");
            break;
    }
}
```

```

        case STRAW_LID:

            System.out.println("Reinserting the straw...");

            break;

        case CORK:

            System.out.println("Pushing the cork back in...");

            break;

    }

}

@Override

public String toString() {

    return String.format("Bottle[shape=%s, color=%s, material=%s, capacity=%.0fml, capType=%s]",

        shape, color, material, capacityML, capType);

}

}

// File: WaterBottle.java

/**
 * WaterBottle inherits from Bottle and adds water-specific behavior.
 * For example, you can track current fill level (in mL),
 * whether it has an infuser, insulation, etc.
 */

public class WaterBottle extends Bottle {

    private double currentVolumeML;    // how much liquid is currently inside

    private boolean insulated;         // double-wall insulation?

    private boolean hasFilter;         // does it have a built-in filter?

    // Constructor

    public WaterBottle(String shape, String color, String material, double capacityML,

        CapType capType, boolean insulated, boolean hasFilter) {

        super(shape, color, material, capacityML, capType);

        this.insulated = insulated;

        this.hasFilter = hasFilter;

        this.currentVolumeML = 0.0;    // starts empty

    }

    // Encapsulated getters/setters for new fields

```

```

public double getCurrentVolumeML() {
    return currentVolumeML;
}

public void setCurrentVolumeML(double currentVolumeML) {
    if (currentVolumeML < 0 || currentVolumeML > getCapacityML()) {
        throw new IllegalArgumentException("Volume out of range");
    }
    this.currentVolumeML = currentVolumeML;
}

public boolean isInsulated() {
    return insulated;
}

public void setInsulated(boolean insulated) {
    this.insulated = insulated;
}

public boolean hasFilter() {
    return hasFilter;
}

public void setHasFilter(boolean hasFilter) {
    this.hasFilter = hasFilter;
}

// Fill the bottle to a given volume (in mL)
public void fill(double amountML) {
    if (amountML <= 0) {
        throw new IllegalArgumentException("Fill amount must be positive");
    }
    double newVolume = currentVolumeML + amountML;
    if (newVolume > getCapacityML()) {
        currentVolumeML = getCapacityML();
        System.out.println("Bottle is now full at " + currentVolumeML + " mL.");
    } else {
        currentVolumeML = newVolume;
        System.out.println("Filled " + amountML + " mL. Current volume: " + currentVolumeML + "
mL.");
    }
}

```

```

    }

}

// Drink (remove) a certain amount of water
public void drink(double amountML) {
    if (amountML <= 0) {
        throw new IllegalArgumentException("Drink amount must be positive");
    }
    if (amountML > currentVolumeML) {
        System.out.println("Not enough water. Drinking remaining " + currentVolumeML + " ml.");
        currentVolumeML = 0;
    } else {
        currentVolumeML -= amountML;
        System.out.println("Drank " + amountML + " ml. Remaining: " + currentVolumeML + " ml.");
    }
}

@Override
public String toString() {
    return String.format("WaterBottle[%s, currentVolume=%.0fml, insulated=%b, hasFilter=%b]",
        super.toString(), currentVolumeML, insulated, hasFilter);
}
}

// File: TestWaterBottle.java
public class TestWaterBottle {
    public static void main(String[] args) {
        // Create a non-insulated plastic water bottle with a screw-on cap, 750 ml capacity
        WaterBottle wb = new WaterBottle(
            "Cylindrical",
            "Transparent Blue",
            "Plastic",
            750.0,
            Bottle.CapType.SCREW_ON,
            false,    // not insulated
            false     // no filter
        );
    }
}

```

```

        System.out.println(wb);

        // Try opening, filling, drinking, and closing:

        wb.openCap();

        wb.fill(500);           // fill 500 ml

        wb.drink(200);          // drink 200 ml

        wb.drink(400);          // attempt to drink more than available (should empty)

        wb.closeCap();

        System.out.println(wb);

        // Create an insulated stainless-steel bottle with a flip-top and filter, 1000 ml capacity
        WaterBottle wb2 = new WaterBottle(

            "Cylindrical",

            "Matte Black",

            "Stainless Steel",

            1000.0,

            Bottle.CapType.FLIP_TOP,

            true,           // insulated

            true            // has filter

        );

        System.out.println("\n" + wb2);

        wb2.openCap();

        wb2.fill(1200);        // attempt to overfill

        wb2.drink(300);

        wb2.closeCap();

        System.out.println(wb2);

    }
}

```

Create random game

```

import java.util.*;

import java.util.concurrent.ConcurrentHashMap;

/** Represents a player in the system. */
public class Player {

    private final String id;

```

```

    public Player(String id) { this.id = id; }

    public String getId()    { return id; }
}

/** Types of games we support, each knows how many players it needs. */
enum GameType {

    SNAKE(1),

    LUDO(4),

    TICTACTOE(2),

    CHESS(2);

    private final int requiredPlayers;

    GameType(int req) { this.requiredPlayers = req; }

    public int getRequiredPlayers() { return requiredPlayers; }
}

/** Common Game interface every concrete game implements. */
interface Game {

    GameType getType();

    /** Kick off the game with exactly getType().getRequiredPlayers() players. */
    void start(List<Player> players);
}

/** Factory to produce new Game instances on demand. */
class GameFactory {

    public static Game createGame(GameType type) {

        switch(type) {

            case SNAKE:      return new SnakeGame();

            case LUDO:       return new LudoGame();

            case TICTACTOE:  return new TicTacToeGame();

            case CHESS:      return new ChessGame();

            default:         throw new IllegalArgumentException();

        }

    }

}

/** Manages incoming players, assigns them randomly to a game type, and starts the game when full. */

```

```

class GameManager {

    private final Map<GameType,List<Player>> waiting
        = new ConcurrentHashMap<>();
    private final Random rnd = new Random();

    public GameManager() {
        for(GameType gt : GameType.values()) {
            waiting.put(gt, new ArrayList<>());
        }
    }

    /** Call this when a new player arrives. */
    public void playerArrives(Player p) {

        // pick a random game type
        GameType type = GameType.values()[rnd.nextInt(GameType.values().length)];
        List<Player> bucket = waiting.get(type);

        synchronized(bucket) {

            bucket.add(p);

            if (bucket.size() == type.getRequiredPlayers()) {

                // time to start
                List<Player> players = new ArrayList<>(bucket);
                bucket.clear();

                Game g = GameFactory.createGame(type);
                g.start(players);

            }

        }

    }

}

class SnakeGame implements Game {

    public GameType getType() { return GameType.SNAKE; }

    public void start(List<Player> players) {

        // players.size() == 1

        System.out.println("Starting Snake for " + players.get(0).getId());

        // ... actual snake logic ...
    }

}

```

```

    }
}

class LudoGame implements Game {

    public GameType getType() { return GameType.LUDO; }

    public void start(List<Player> players) {

        // players.size() == 4

        System.out.println("Starting Ludo for " + players);

        // ... actual ludo logic ...

    }
}

// similarly:

class TicTacToeGame implements Game { /* ... */ }

class ChessGame      implements Game { /* ... */ }

```

Maximum Product Subarray

```

import java.util.Arrays;

public class Solution {

    /**
     * Returns the contiguous subarray that has the maximum product.
     * Uses O(n) time and O(1) extra space.
     *
     * @param nums input array of integers (may include negatives and zeros)
     * @return the subarray (as a new int[]) whose product is maximal
     */

    public int[] maxProductSubarray(int[] nums) {

        int n = nums.length;

        if (n == 0) return new int[0];

        // prev_max and prev_min track the max/min product ending at previous index

        int prevMax = nums[0], prevMin = nums[0];

        // start indices for those products

        int prevMaxStart = 0, prevMinStart = 0;

        // global best
    }
}

```

```

int globalMax = nums[0];
int globalStart = 0, globalEnd = 0;

for (int i = 1; i < n; i++) {
    int x = prevMax * nums[i];
    int y = prevMin * nums[i];
    int z = nums[i];

    // Determine current max and its start index
    int curMax, curMaxStart;
    if (z >= x && z >= y) {
        curMax = z;
        curMaxStart = i;
    } else if (x >= y) {
        curMax = x;
        curMaxStart = prevMaxStart;
    } else {
        curMax = y;
        curMaxStart = prevMinStart;
    }

    // Determine current min and its start index
    int curMin, curMinStart;
    if (z <= x && z <= y) {
        curMin = z;
        curMinStart = i;
    } else if (x <= y) {
        curMin = x;
        curMinStart = prevMaxStart;
    } else {
        curMin = y;
        curMinStart = prevMinStart;
    }

    // Update global best subarray if needed
    if (curMax > globalMax) {
        globalMax = curMax;

```

```

        globalStart = curMaxStart;

        globalEnd = i;
    }

    // Move current to prev for next iteration
    prevMax = curMax;
    prevMin = curMin;
    prevMaxStart = curMaxStart;
    prevMinStart = curMinStart;
}

// Extract the subarray from globalStart to globalEnd inclusive
return Arrays.copyOfRange(nums, globalStart, globalEnd + 1);
}

// Example usage
public static void main(String[] args) {
    Solution sol = new Solution();

    int[] arr1 = {2, 3, -2, 4};
    // max product = 6 from subarray [2,3]
    System.out.println(Arrays.toString(sol.maxProductSubarray(arr1))); // [2, 3]

    int[] arr2 = {-2, 0, -1};
    // max product = 0 from subarray [0]
    System.out.println(Arrays.toString(sol.maxProductSubarray(arr2))); // [0]

    int[] arr3 = {-2, 3, -4};
    // max product = 24 from subarray [3, -4]
    System.out.println(Arrays.toString(sol.maxProductSubarray(arr3))); // [3, -4]

    int[] arr4 = { -1, -3, -10, 0, 60 };
    // max product = 60 from [60]
    System.out.println(Arrays.toString(sol.maxProductSubarray(arr4))); // [60]

    int[] arr5 = { -2, -3, 0, -2, -40 };
    // max product = 80 from [-2, -40]

```

```
        System.out.println(Arrays.toString(sol.maxProductSubarray(arr5))); // [-2, -40]
    }
}
```

Unique Object

```
import java.util.concurrent.atomic.AtomicLong;

public class UniqueObject implements Cloneable {

    // A thread-safe global counter for issuing unique IDs
    private static final AtomicLong ID_GENERATOR = new AtomicLong(0);

    // The unique ID of this instance
    private final long id;

    // Example payload field
    private String data;

    /**
     * Default constructor – issues a fresh ID.
     */
    public UniqueObject() {
        this.id = ID_GENERATOR.incrementAndGet();
    }

    /**
     * Copy constructor – issues a new ID, but copies other fields.
     */
    public UniqueObject(UniqueObject other) {
        this.id = ID_GENERATOR.incrementAndGet();
        this.data = other.data;
    }

    /**
     * Clone override – uses our copy-constructor so the clone
     * gets a brand-new ID.
     */
    @Override
```

```
public UniqueObject clone() {
    return new UniqueObject(this);
}

public long getId() {
    return id;
}

public String getData() {
    return data;
}

public void setData(String data) {
    this.data = data;
}

@Override
public String toString() {
    return "UniqueObject{id=" + id + ", data=" + data + "}";
}

// Simple demo
public static void main(String[] args) {
    UniqueObject original = new UniqueObject();
    original.setData("hello");

    // "Copy" via copy constructor
    UniqueObject copy1 = new UniqueObject(original);

    // "Copy" via clone()
    UniqueObject copy2 = original.clone();

    System.out.println(original); // e.g. id=1
    System.out.println(copy1);    // e.g. id=2
    System.out.println(copy2);    // e.g. id=3

    // Assignment of reference does NOT create a new object:
```

```
        UniqueObject alias = original;

        System.out.println(alias);    // same id as original
    }
}
```

Thread creation

```
import java.util.concurrent.*;

// 1) Extend Thread
class ThreadExt extends Thread {
    private final String name;

    public ThreadExt(String name) {
        this.name = name;
    }

    @Override
    public void run() {
        System.out.printf("[%s] running in %s\n", name, Thread.currentThread().getName());
    }
}

// 2) Implement Runnable
class RunnableImpl implements Runnable {
    private final String name;

    public RunnableImpl(String name) {
        this.name = name;
    }

    @Override
    public void run() {
        System.out.printf("[%s] running in %s\n", name, Thread.currentThread().getName());
    }
}

// 3) Implement Callable
class CallableImpl implements Callable<String> {
    private final String name;

    public CallableImpl(String name) {
        this.name = name;
    }
}
```

```

    }

    @Override
    public String call() throws Exception {
        String msg = String.format("[%s] computed in %s", name, Thread.currentThread().getName());
        System.out.println(msg);
        return msg;
    }
}

public class ThreadCreationDemo {
    public static void main(String[] args) throws Exception {
        // 1) Using Thread subclass
        ThreadExt t1 = new ThreadExt("ThreadExt");
        t1.start();

        // 2) Using Runnable
        Thread t2 = new Thread(new RunnableImpl("RunnableImpl"));
        t2.start();

        // 3) Using Callable with FutureTask
        CallableImpl callable = new CallableImpl("CallableImpl");
        FutureTask<String> future = new FutureTask<>(callable);
        Thread t3 = new Thread(future);
        t3.start();

        // Optionally get the result of the Callable
        String result = future.get(); // blocks until call() completes
        System.out.println("Result from callable: " + result);

        // Clean up: you could also use an ExecutorService
        ExecutorService exec = Executors.newSingleThreadExecutor();
        Future<String> f2 = exec.submit(new CallableImpl("Callable-via-Executor"));
        System.out.println("Executor result: " + f2.get());
        exec.shutdown();
    }
}

```